

C Programming

A practical study reference with foundational concepts, examples, and common review questions.

1. Compilation and Types

C is a compiled procedural language used in systems and embedded software. A toolchain preprocesses, compiles, and links translation units. Header files declare interfaces; source files define them. `main` is the conventional entry point. Compiler flags and the selected C standard affect available features.

Types include integers, floating-point values, arrays, structures, unions, enums, and pointers. Integer widths can vary by platform; `stdint.h` defines fixed-width integer types. Use `size_t` for sizes and indexes, and check conversions, bounds, and overflow.

2. Arrays, Pointers, Strings

An array stores contiguous elements. In many expressions it converts to a pointer to its first element, but an array is not itself a pointer. Pointer arithmetic is valid only within an array and one position past its end. C passes arguments by value; pass a pointer to allow a function to modify a caller's object.

A C string is a character array terminated by `'\0'`; storage must include the terminator. Validate buffer lengths. Undefined behavior, such as out-of-bounds access or signed overflow, means the language imposes no requirements on the result.

```
int sum(const int *v, size_t n) {
    int total = 0;
    for (size_t i = 0; i < n; ++i) total += v[i];
    return total;
}
```

3. Memory and Files

Automatic local objects typically have block lifetime; static objects last for the program duration. `malloc` allocates dynamic storage and `free` releases it. Check allocation results and prevent leaks, double frees, and use-after-free. Guard against overflow in `count * sizeof(element)`.

Use FILE functions such as `fopen`, `fgets`, and `fclose` and check their return values. Use compiler warnings, sanitizers, and static analysis. Do not use untrusted text as a `printf` format string; validate all external lengths and inputs.

4. Common Questions

A pointer stores an address and enables indirect access. Stack and heap describe common storage strategies, but are not a complete language guarantee. A memory leak is allocated storage that can no longer be released by the program. `const` can prevent modification through a particular pointer or reference.